# BRIEF INTRODUCTION TO REGULAR EXPRESSIONS IN C#

Author: Héctor Fortún Castañón.

Lic. Cybernetics-Mathematics
University of Havana

**Content:**

What are regular expressions? ..... 3

How can I validate a string with regular expressions in C# language? ..... 4

How to build a regular expression pattern in C#? ..... 5

1. Use of square brackets (Example 1). ..... 5

The negation within square brackets. ..... 9

2. Use of curly braces (Example 2). ..... 9

3. The use of the backslash \ ..... 9

\t — TAB key (Tab Character). ..... 11

\r — "Car Return" or "Car Return" (CR) character. ..... 11

\n — "New Line" or Line Feed (LF) character. ..... 11

\a — "Beep" character, or bell. ..... 11

\e — "Esc" or "Escape" character ..... 11

\f — "Page Break" or Form Feed (<FF>) character. ..... 11

\v — Vertical tab (<VT>) ..... 11

\xNN — ASCII hexadecimal character ..... 11

\uNNNN — Unicode character ..... 11

\d — Decimal digit. ..... 11

\D — Any character other than a decimal digit. ..... 11

\w — Alphanumeric character (alphabetical or numeric including ..... 12

the underlining). ..... 12

\W — NON-alphanumeric character. ..... 12

\s — Blank space. ..... 12

\S — Different character of a space. ..... 12

\b — Word boundary. ..... 12

\Z — End of the string ..... 13

\B — Not a word boundary. ..... 13
\A — Start of the string ..... 14
\G — Coincidences in contiguous areas of the chain under analysis ..... 14
\p — Unicode Character Class or Named Block ..... 14
General Unicode Categories (Microsoft table) ..... 14
Named Blocks (Microsoft Table) ..... 15
\P — Unicode Negation Character Class or Named Block ..... 17
4. Alternation Constructions (Use of logical OR "|") ..... 18
5. Brackets "[]" in more detail ..... 19
6. The parentheses "()" ..... 20
7. The character "." ..... 22
8. The question mark "?" ..... 22
9. The format (?:pattern) ..... 23
10. The format (?=pattern) ..... 24
11. The format (?!pattern) ..... 24
12. The Quantifiers ..... 25
Expansive and non-expansive quantifiers ..... 25
Some examples of the use of quantifiers (Microsoft) ..... 27
13. Regular Expression Utilities ..... 28
14. The Replace method of the Regex class. ..... 28
Replace method ..... 29
15. Excluding Character Groups from a Character Class ..... 35
16. Exercises to Solve ..... 35
16.1 - Exercise 1 ..... 35
16.2 - Exercise2 ..... 35
16.3 - Exercise3 ..... 35
16.4 - Exercise4 ..... 35
16.5 - Exercise5 ..... 36
16.6 - Exercise6 ..... 36
16.7 - Exercise7 ..... 36
16.8 - Exercise8 ..... 36
16.9 - Exercise9 ..... 36
Final summary ..... 37

## What are regular expressions?

When you observe certain strings of characters (strings), such as a decimal number, you notice that there is a certain law of formation, in this case, a string of decimal digits followed by a decimal point and then another string of decimal digits In another example, when we observe a date, we quickly imagine a string of the form dd/mm/yyyy or mm/dd/yyyy where dd represents the day, mm the month and yyyy the 4 digits of the year, well, we are in the presence of a certain regularity in its law of formation, from which it is inferred that if a pattern (pattern in English literature) is created with a certain syntax that can describe said law of formation, then this would considerably facilitate some processes such as validations , replacements etc. within computational processes, and exactly this is what has been called regular expressions which exist in various programming languages, such as C#, C++, Visual Basic, Php, Oracle, MySQL, and many others.

Regular expressions are validation patterns that are represented by a parameterized string of characters where each character has a meaning within the substring identification process, for this reason, when we define a certain pattern, it could accept or not, a string of input characters.

There are two general categories of regular expressions: POSIX and PERL.

In our case we will focus on the second group (PERL), which is the family of the .NET regular expression engine used by Visual Studio's C#. If the reader wants to go deeper theoretically into this topic, he or she will be able to find a lot of information published on the Internet, although the Visual Studio HELP of Microsoft will also be a great reference.

To the extent that this topic is theoretically delved into, the reader will quickly realize the code savings that can be obtained in the validation, replacement and other processes with the use of these expressions.

The fundamental objective of this document is to allow the neophyte in this subject to quickly break inertia in order to be able to use these techniques in programming under Microsoft's C# language. It is also convenient to clarify that for this purpose we created this simple utility program <RegexTest> that will allow you to process the regular expressions that you define, generating the C# code corresponding to the validations, replacements and SPLITS that you want for your program, which will save you a lot of programming time in your task.

## How can I validate a string with regular expressions in C# language?

Very simple, there is a class called Regex that will allow us to perform these actions, follow the following steps:

- Declare a sentence:
  using System.Text.RegularExpressions; `in the program header.`

- Instantiate an object with this class by specifying the validation pattern in the constructor in the general format shown below:

  ```
  RegexRegexObject = new Regex(@"validation pattern");
  ```

  * It is advisable to use the @ at sign for the **string** of the validation pattern so that you can specify some special characters that may be required.

Let's observe below a validation made using the ObjectRegex object instantiated in the previous step and using the **IsMatch** boolean method that checks if the input string that appears in the Text property of the control named TextBox1 without the spaces on the left or right, meets the requirements given in the pattern:

```
if (!RegexObject.IsMatch(TextBox1.Text.Trim()))
{
MessageBox.Show("Bad String", "<ERROR>", MessageBoxButtons.OK,
MessageBoxIcon.Error);
}
else
{
MessageBox.Show("Correct String", "<OK>", MessageBoxButtons.OK,
MessageBoxIcon.Information);
}
```

## How to build a regular expression pattern in C#?

Given that the number of existing parameters is quite extensive, and as always happens, there will be some validations that will interest us more than others, since we will focus our attention on the most used ones which will be explained through practical examples, which is the best way to capture quickly any concept. By now we will show several practical examples that will help you quickly grasp the concepts expressed.

### 1. Use of square brackets (Example 1).

We have purposely started with the use of square brackets and their meaning within a pattern or regular expression pattern.

Each square bracket of the form [pattern_string] represents a character position in the input string being analyzed, and pattern_string tells <what characters> can appear in that position, like this, for example:

[13][5-9][ABX] Indicates that <trio of characters> are accepted whose first character must be a 1 or a 3 (nothing more; note that inside the brackets the characters are not separated by commas, because the comma would be one more accepted character), than the second character of the string. input must be any digit between 5 and 9, and the third character of the input string must be an A, a B or an the first ends, then continue looking ahead looking if there are more shortlists that meet the condition, as in the second and third examples.

From the above it is inferred that the following strings will be accepted for this pattern:

37X==> Accepted string with number of matches equal to 1.

29B15A==> Accepted string with number of matches equal to 1 (15A), because in 29B the first character is neither 1 nor 3.

15A38X16Z==> Accepted string with number of matches equal to 2,
Well, the last shortlist has the first two characters right.
but the Z is not in the range specified by the pattern
entrance.

25A==> Unaccepted string with equal number of matches

to 0 because the first character is neither 1 nor 3.

*** It is necessary to clarify that the string that represents the validation pattern does not**

**It must contain spaces at the beginning or end as this can cause subtle errors.**

If we now assume the following pattern:

[0-5]

This means that each character in the input string can only

contain digits from 0 to 5 only so that there is a match in that position, for this reason:

The string "14205" has 5 matches and is accepted given that at least one match exists.

The string "14905" has 4 matches and is accepted given that at least one match exists.

The string "78" has no matches and is therefore not accepted.

**Note:**

**We suggest that you try these examples with the accompanying utility program "RegexTest".**

After having seen the first type of regular expression pattern specification, you may be saying to yourself, everything is very nice, but since I do it by program, well, then we will dedicate a few paragraphs to this very important topic. and you will see that you will understand everything easily, since this document is fundamentally oriented towards practice.

Firstly, it must be taken into consideration that there is a class in C# called **Regex** that the programmer can use to instantiate regular expression objects, it requires that the pattern be specified as a constructor parameter. Remember to specify the statement at the beginning of the program:

```
using System.Text.RegularExpressions;
```

Example:

```
Regex RegexObject = new Regex(@"[0-5]");
```

*** In this example, an object called RegexObject has been instantiated whose pattern tells us that the character strings containing at least one digit from 0 to 5 will be accepted. In other words, this indicates that every time reference is made to`RegexObject` we will be looking for digits from 0 to 5 within the string being analyzed.**

For what was said above, if we have the string variable Var = "12434", the following C# code could be written in order to validate it with the previous pattern and referring to the object instantiated in the previous paragraph **`(RegexObject)`**:

```
 if (RegexObject.IsMatch(Var))
      {
      // It should be noted that when referencing RegexObject we're
      // telling the regular expression engine that our pattern
      // is [0-5]as we define it in the instantiation of the object.
      // There is a match at least once in Var and the number of matches
      // remains in the integer variable <Coincidencias> whose value is
      // returned by the <Count> property of the <Matches> method.

      int Matches = RegexObject.Matches(Var).Count;

       // The final result is Matches = 5
          . . .
      }

else
      {
      // Here you enter if there was absolutely no match
          // and the code corresponding to such condition will be written.
          . . .
      }
```

There is another way to invoke the IsMatch and Matches methods without needing to instantiate an object and that is that the Regex class also has static methods (called with the name of the class itself, Regex) that receive two parameters; The first contains the variable to be analyzed and the second the validation pattern, in this way:

```
ObjectRegex.IsMatch(Var)
```

would be equivalent to the expression:

```
Regex.IsMatch(Var, @"[0-5]")
```

and to determine matches:

```
ObjectRegex.Matches(Var).Count;
```

would be equivalent to the expression:

```
Regex.Matches(Var, @"[0-5]").Count;
```

Based on what was said above, the code of the previous example would look like this under the new format:

```
if(Regex.IsMatch(Var,@"[0-5]"))
      {
      int Matches = Regex.Matches(Var,@"[0-5]").Count;
      . . .
      }
else
      {
      // Here you enter if there was absolutely no match
      // and the code corresponding to such condition will be written.
      . . .
      }
```

* It will be observed that the @ symbol is being used in the character strings, in this way escape sequences such as slash and others may appear within the string in question.

From now on we will refer to either of the above two formats and you should be familiar with them.

After having seen everything previously explained, we will see another regular expression format, it is the use of the characters ^ and $ at the beginning and at the end of the pattern, that is, if we write`@"^[0-5]$"`We are saying that the analysis will necessarily be done from the beginning of the chain ("**^**") and that said string will be composed only of one character that must be a digit between 0 and 5 which will be implied by the appearance of the symbol ("**$**"). In other words, we are saying that a single character input is expected and that it must be a digit between 0 and 5. (Detail: $ also accepts a line feed \n at the end of the string; for a strict validation you can use \A at the beginning and \z at the end.)

When the brackets are included between these two special symbols (**^**and**$**) Each pair of square brackets represents a character with mandatory presence in the input string, which must also be in the range specified within said square bracket.

As an example of what was said above if we assume the pattern:

```
@"^[0-5][8][479]$"
```

It is assumed that only strings of three digits required where the first character must be a digit from 0 to 5, the second a 8 and the third a digit that must be a 4, a 7 or a 9, in this way the following strings would be accepted:

- 587
- 084
- 489
- 184

In contrast, the following would not be accepted:

- 5875 ==> Has more than three characters
- 587084 ==> Has more than three characters
- 579 ==> The second digit is not an 8
- 111 ==> The second and third digit is out of range

Now, possibly the reader is analyzing other alternatives, such as, for example, what would happen if we remove the ^ symbol from the pattern, that is, [0-5][8][479]$, well, in this In this case we are telling the regular expression parsing engine that the parsing will not necessarily begin at the beginning of the string but rather the last three characters would be chosen.
Considering the above, the string "587084" would give us that the match is the substring "084", and if we put the string "587" then the match would be that same string.

And what would happen if we remove the $ symbol from the previous pattern, that is, [0-5][8][479],
very simple, now we are telling the regular expression engine that all substrings within the input string that meet the condition given by said pattern will be accepted, for example:
The string "587084" would give us two matches, the substring "587" and the substring "084" since the absence of the $ symbol is removing the right dimension.

**The negation within square brackets.**

It is possible to negate specifications given within a square bracket and the format is as follows:

[^specification]

That is, if you specify [^6], this means that any character that is not a digit 6 is accepted in that position. As a practical example, if we do not want any decimal digit to exist in that position we would write [^0-9].

## 2. Use of curly braces (Example 2).

The curly braces are interpreted as the number of characters to parse in the input string, like this, for example:

The pattern "[5-6]{3}" tells us that only substrings whose digits are between 5 and 6 and that must be exactly 3 characters long will be accepted, thus, the string "565084665" would give us two matches, the substrings "565" and the substring "665" since "084" is out of range.
It should be noted that the pattern "[5-6]{3}" is equivalent to the pattern "[5-6][5-6][5-6]" written in reduced form.

## 3. The use of the backslash \

The character "\" is frequently used to define different escape sequences with a given meaning, for example, when you want to indicate that in a given position of the input string there must be a decimal digit that until now we have been symbolizing by brackets"[0-9]", since this can also be indicated by a**\d,** for this reason, a pattern of the following form:

**"^[0-9]{3}\.[0-9]{2}$"would be equivalent to ==> "^\d{3}\.\d{2}$"**

It is telling us that only 6-character strings representing numbers with three integer digits, followed by a decimal point and two decimal digits, are accepted. Example: 013.22 (see the note on \d below).

As an example, if we want to validate strings that must have three decimal digits at the beginning followed by a hyphen and 4 decimal digits, we would write the following pattern:

**^\d{3}\-\d{4}$**

which is also equivalent to this other pattern under the use of square brackets:

**^[0-9]{3}[-][0-9]{4}$**

Some examples:

| Accepted Strings | Unaccepted Strings |
|---|---|
| 832-0899 | 8323-0899 (has more than 3 digits before the hyphen) |
| 261-9130 | 261-91303 (has more than 4 digits after the dash) |
| 838-0047 | 053-449005 (has more than 4 digits after the dash) |

The "\" metacharacter indicates that the next character is a special character, a literal, or an escape sequence.

For example, if the character "n" is used within the pattern, it represents a letter n in the corresponding position, while if "\n" is written the meaning is completely different since it defines a line change character (line feed).

When you want to specify a non-ASCII representative character through its internal hexadecimal value, you can use an escape sequence with the following general format:

**\xNN**

where NN represents two hexadecimal digits, from 00 to FF (the ASCII table and the Latin characters that follow it, such as ñ = \xF1).

On the other hand, if you wish to specify any UNICODE character, the following format is used:

**\uNNNN**

where NNNN represents four hexadecimal digits (the UTF-16 code of the character).

If you write:

**\cx**

this is interpreted as a Ctrl-X since **\c** is associated with the key**CTRL**.

**Important note:**

When you want to include the backslash ("\") as a character within the pattern we have to write "\\" and it stops being a metacharacter and becomes an ordinary character within the string.

Below we will see a series of escape sequences that can be used within regular expressions:

==========

**\t – TAB key (Tab Character).**

Example:

| Accepted Strings | Unaccepted Strings |
|---|---|
| 3<tab>5 | 47<tab>56 |
| 8<tab>1 | 3<tab>55 |
| 9<tab>4 | 222<tab>888 |

**\r – "Car Return" or "Car Return" (CR) character.**
**\n – "New Line" or Line Feed (LF) character.**
**\a – "Beep" character, or bell.**
**\e – "Esc" or "Escape" character**
**\f – "Page Break" or Form Feed (<FF>) character.**
**\v – Vertical tab (<VT>)**
**\xNN – ASCII hexadecimal character**
**\uNNNN – Unicode character**

Idem as above but to represent characters**Unicode**,for example "\u00A2" represents the cents symbol.
* Not all Regular Expression engines support Unicode.
The .NET Framework does this, but others may not support it.

**\d – Decimal digit.**
*** Already seen in the previous examples. Note that in .NET \d accepts any Unicode decimal digit, not only 0 to 9 (for example, also the Arabic digits ٠١٢٣); if you want only the digits 0 to 9, use [0-9] or the RegexOptions.ECMAScript option.

**\D – Any character other than a decimal digit.**

*** It is the inverse of \d

### \w — Alphanumeric character (alphabetical or numeric including the underlining).

In .NET it accepts any letter (including ñ, accented vowels and ü), any decimal digit and the underscore.

Only if the RegexOptions.ECMAScript option is used does it correspond to the pattern [A-Za-z0-9_], that is, to the English alphabet, which does not include ñ, accented letters, etc.

Therefore, without that option, \w already works for words written in Spanish.

### \W — NON-alphanumeric character.

* It is the inverse of \w, which means it can be any character other than letters, numbers and underscores (of any alphabet)

### \s — Blank space.

*** Represents any white-space character: the space, the tab (\t), line breaks (\n and \r), form feed (\f) and vertical tab (\v), among others.

### \S — Different character of a space.

*** Represents any character that is not a white-space character.

### \b — Word boundary.

If we take the following pattern:**er\b**

\b marks a word boundary, that is, the beginning or the end of a word. That is why er\b accepts the strings that contain a word ending in "er". The examples keep the Spanish words of the original manual, because what matters is the pattern: "amanecer", "tener", etc. If it is written in front, \ber, it looks for words that begin with "er", such as "error".

Example:

| Accepted Strings | Unaccepted Strings |
|---|---|
| amanecer | error |
| prevalecer | persistir |
| mover | mover_ |

However, if the following pattern were set:**^er\b$**

The only accepted string would be "er" which is very different, since the ^ metacharacter requires that only the beginning of the string be before "er" and the $ that only the end be after it.

**Disclaimer:**
The escape sequences \b, \B, \G, \Z, \z, \A, ^, and $ that are explained in this document are known as DELIMITERS or also as "Zero Width Atomic Assertions" which make that the match is fulfilled or not depending on the position of the substring within the string, but they do not cause the regular expression engine to advance in its analysis of the string or consume characters.

## \Z — End of the string

If we take the following pattern:**er\Z**

This indicates that the accepted string will be the one that ends in "er": \Z marks the end of the whole string (or the position just before a final line feed \n).

Example:

| Accepted Strings | Unaccepted Strings |
|---|---|
| amar comer | error |
| prevalecer | persistir |
| partir amar mover | mover_ |

Note that \Z only looks at the end of the whole string: in "comer\nhola" there is no match even though the first line ends in "er". If you do not want to accept the final line feed either, use \z (lowercase), which requires the exact end of the string.

## \B — Not a word boundary.

If we take the following pattern:**er\B**
\B is the opposite of \b: the position must NOT be a word boundary. That is why er\B accepts an "er" that is not at the end of a word, that is, followed by another letter, as in "error". "_error" or "persistir" would also be accepted, because in both the "er" is followed by another letter.

Example:

| Accepted Strings | Unaccepted Strings |
|---|---|
| error errata<br>(In this case there are 2 matches) | mover |
| erradicar<br>(In this case there is 1 match) | comer y beber |
| error errata,erroneo:erratarrorr<br><br>(In this case there are 5 matches) | repeler |

In the first column the "er" highlighted in yellow cause a match. If the pattern ^er\B were used, only an "er" at the beginning of the string and followed by another letter would be accepted, as in "error".

### \A — Start of the string

If we take the following pattern: \Aer
This indicates that the strings that begin with "er" will be accepted: \A marks the start of the whole string (\A\er must not be written, because \e represents the ESC character).

For example, if we take the string "error errata Ernesto" causes 1 match only, which occurs in the prefix of the first word (highlighted in yellow).

### \G — Coincidences in contiguous areas of the chain under analysis

If we take the following pattern:**\G\(\d\)**

This indicates that the accepted strings will be those that contain the first appearance of contiguous substrings that follow this formation law defined by a parenthesis, a digit and a closed parenthesis and that appear in turn as a prefix of the string under analysis.

For example, if we take the string "(1)(9)(5)[2](4)(7)" the matches appear highlighted in yellow, however, if we take the string "[2](5)(2)" this does not cause matches because (5)(2) do not appear as prefixes of the chain under analysis.

### \p — Unicode Character Class or Named Block

Matches any single character that belongs to the general Unicode category or the named block specified by name.

There are general categories of international character sets that group a set of specific categorized characters which are designated by a name that in regular expressions must be enclosed in braces, that is, under the format {category_name} or also as {block_with_name}.

When you want the pattern to only recognize symbols belonging to the previous categories or named blocks, then you must use the \p escape sequence in the following format:

\p{category_name}
\p{block_with_name}

==========
==========

Below, we show in detail the tables extracted from the Internet (Microsoft) for both cases:

### General Unicode Categories (Microsoft table)

| Category | Description |
|---|---|
| **Lu** | Capital letter |
| **Ll** | Lowercase letter |

| | |
|---|---|
| **Lt** | Letter, capital initial |
| **Lm** | Letter, modifier |
| **It** | Letter, other |
| **l** | All letter characters. This includes the characters Lu, Ll, Lt, Lm and Lo. |
| **mn** | Mark, no spaces |
| **Mc** | Brand, with spaces and combination |
| **I** | Brand, inclusion |
| **M** | All diacritical marks. This includes the categories Mn, Mc and Me. |
| **nd** | Number, decimal digit |
| **Nl** | Number, letter |
| **No** | number, other |
| **N** | All the numbers. This includes categories Nd, Nl and No. |
| **PC** | Punctuation, connector |
| **P.S** | Score, score |
| **Ps** | Score, open |
| **Pe** | Score, close |
| **Pi** | Punctuation, opening quote (can behave like Ps or Pe, depending on usage) |
| **Pf** | Punctuation, closing quote (can behave like Ps or Pe, depending on usage) |
| **po** | Score, other |
| **Q** | All punctuation marks. This includes the categories Pc, Pd, Ps, Pe, Pi, Pf and Po. |
| **YE** | Symbol, mathematician |
| **Sc** | Symbol, badge |
| **Sk** | Symbol, modifier |
| **SW** | Symbol, other |
| **Yes** | All symbols. This includes the Sm, Sc, Sk and So categories. |
| **Zs** | Separator, space |
| **Zl** | Separator, line |
| **Zp** | Separator, paragraph |
| **Z** | All separator characters. This includes categories Zs, Zl and Zp. |
| **DC** | Other, control |
| **CF** | Other, format |
| **cs** | Other, substitute |
| **Co** | Other, private use |
| **cn** | Other, unassigned (no character has this property) |
| **c** | All control characters. This includes categories Cc, Cf, Cs, Co and Cn. |

It is possible to obtain the Unicode category of any character by passing said character to the method **GetUnicodeCategory**.

Example:

```
Regex.Escape(char.ToString()),Char.GetUnicodeCategory(char));
```

Let's see below the table corresponding to named blocks:

## Named Blocks (Microsoft Table)

The .NET Framework provides the named blocks shown in the following table. The set of supported named blocks is based on Unicode 4.0 and Perl 5.6.

| Code interval | Block name |
|---|---|
| 0000 - 007F | **IsBasicLatin** |
| 0080 - 00FF | **IsLatin-1Supplement** |
| 0100 - 017F | **IsLatinExtended-A** |
| 0180 - 024F | **IsLatinExtended-B** |
| 0250 - 02AF | **IsIPAExtensions** |
| 02B0 - 02FF | **IsSpacingModifierLetters** |
| 0300 - 036F | **IsCombiningDiacriticalMarks** |
| 0370 - 03FF | **IsGreek**<br>**O well**<br>**IsGreekandCoptic** |

| | |
|---|---|
| 0400 - 04FF | **IsCyrillic** |
| 0500 - 052F | **IsCyrillicSupplement** |
| 0530 - 058F | **IsArmenian** |
| 0590 - 05FF | **IsHebrew** |
| 0600 - 06FF | **IsArabic** |
| 0700 - 074F | **IsSyriac** |
| 0780 - 07BF | **IsThaana** |
| 0900 - 097F | **IsDevanagari** |
| 0980 - 09FF | **IsBengali** |
| 0A00 - 0A7F | **IsGurmukhi** |
| 0A80 - 0AFF | **IsGujarati** |
| 0B00 - 0B7F | **IsOriya** |
| 0B80 - 0BFF | **IsTamil** |
| 0C00 - 0C7F | **IsTelugu** |
| 0C80 - 0CFF | **IsKannada** |
| 0D00 - 0D7F | **IsMalayalam** |
| 0D80 - 0DFF | **IsSinhala** |
| 0E00 - 0E7F | **IsThai** |
| 0E80 - 0EFF | **Lao Island** |
| 0F00 - 0FFF | **IsTibetan** |
| 1000 - 109F | **IsMyanmar** |
| 10A0 - 10FF | **IsGeorgian** |
| 1100 - 11FF | **IsHangulJamo** |
| 1200 - 137F | **IsEthiopic** |
| 13A0 - 13FF | **IsCherokee** |
| 1400 - 167F | **IsUnifiedCanadianAboriginalSyllabics** |
| 1680 - 169F | **IsOgham** |
| 16A0 - 16FF | **IsRunic** |
| 1700 - 171F | **IsTagalog** |
| 1720 - 173F | **IsHanunoo** |
| 1740 - 175F | **IsBuhid** |
| 1760 - 177F | **IsTagbanwa** |
| 1780 - 17FF | **IsKhmer** |
| 1800 - 18AF | **IsMongolian** |
| 1900 - 194F | **IsLimbu** |
| 1950 - 197F | **IsTaiLe** |
| 19E0 - 19FF | **IsKhmerSymbols** |
| 1D00 - 1D7F | **IsPhoneticExtensions** |
| 1E00 - 1EFF | **IsLatinExtendedAdditional** |
| 1F00 - 1FFF | **IsGreekExtended** |
| 2000 - 206F | **IsGeneralPunctuation** |
| 2070 - 209F | **IsSuperscriptsandSubscripts** |
| 20A0 - 20CF | **IsCurrencySymbols** |
| 20D0 - 20FF | **IsCombiningDiacriticalMarksforSymbols O well IsCombiningMarksforSymbols** |
| 2100 - 214F | **IsLetterlikeSymbols** |
| 2150 - 218F | **IsNumberForms** |
| 2190 - 21FF | **IsArrows** |
| 2200 - 22FF | **IsMathematicalOperators** |
| 2300 - 23FF | **IsMiscellaneousTechnical** |
| 2400 - 243F | **IsControlPictures** |
| 2440 - 245F | **IsOpticalCharacterRecognition** |
| 2460 - 24FF | **IsEnclosedAlphanumerics** |
| 2500 - 257F | **IsBoxDrawing** |
| 2580 - 259F | **IsBlockElements** |
| 25A0 - 25FF | **IsGeometricShapes** |
| 2600 - 26FF | **IsMiscellaneousSymbols** |
| 2700 - 27BF | **IsDingbats** |
| 27C0 - 27EF | **IsMiscellaneousMathematicalSymbols-A** |
| 27F0 - 27FF | **IsSupplementalArrows-A** |
| 2800 - 28FF | **IsBraillePatterns** |

| | |
|---|---|
| 2900 - 297F | IsSupplementalArrows-B |
| 2980 - 29FF | IsMiscellaneousMathematicalSymbols-B |
| 2A00 - 2AFF | IsSupplementalMathematicalOperators |
| 2B00 - 2BFF | IsMiscellaneousSymbolsandArrows |
| 2E80 - 2EFF | IsCJKRadicalsSupplement |
| 2F00 - 2FDF | IsKangxiRadicals |
| 2FF0 - 2FFF | IsIdeographicDescriptionCharacters |
| 3000 - 303F | IsCJKSymbolsandPunctuation |
| 3040 - 309F | IsHiragana |
| 30A0 - 30FF | IsKatakana |
| 3100 - 312F | IsBopomofo |
| 3130 - 318F | IsHangulCompatibilityJamo |
| 3190 - 319F | IsKanbun |
| 31A0 - 31BF | IsBopomofoExtended |
| 31F0 - 31FF | IsKatakanaPhoneticExtensions |
| 3200 - 32FF | IsEnclosedCJKLettersandMonths |
| 3300 - 33FF | IsCJKCompatibility |
| 3400 - 4DBF | IsCJKUnifiedIdeographsExtensionA |
| 4DC0 - 4DFF | IsYijingHexagramSymbols |
| 4E00 - 9FFF | IsCJKUnifiedIdeographs |
| A000 - A48F | IsYiSyllables |
| A490 - A4CF | IsYiRadicals |
| AC00 - D7AF | IsHangulSyllables |
| D800 - DB7F | IsHighSurrogates |
| DB80 - DBFF | IsHighPrivateUseSurrogates |
| DC00 - DFFF | IsLowSurrogates |
| E000 - F8FF | IsPrivateUse or IsPrivateUseArea |
| F900 - FAFF | IsCJKCompatibilityIdeographs |
| FB00 - FB4F | IsAlphabeticPresentationForms |
| FB50 - FDFF | IsArabicPresentationForms-A |
| FE00 - FE0F | IsVariationSelectors |
| FE20 - FE2F | IsCombiningHalfMarks |
| FE30 - FE4F | IsCJKCompatibilityForms |
| FE50 - FE6F | IsSmallFormVariants |
| FE70 - FEFF | IsArabicPresentationForms-B |
| FF00 - FFEF | IsHalfwidthandFullwidthForms |
| FFF0 - FFFF | IsSpecials |

After having seen the previous tables, an example of the use of \p is shown.

```
\p{Sc}\d+\.\d+
```

Accept substrings highlighted in yellow:

23$44.55 €34.68429.22£77.49

since Sc represents the monetary symbols.

**\P — Unicode Negation Character Class or Named Block**

The expression:

\P{name}

matches all characters not included in the name block.

From the above it is inferred that this expression is the negation of \p.

If we take as an example the expression **\P{Sc}** We are saying that all characters will be accepted, except monetary symbols ($,€,£, etc.). It should be noted that this expression is the equivalent of the ^ symbol inside a square bracket.

## 4. Alternation Constructions (Use of logical OR "|")

Alternation constructions create a group of alternatives that will or will not cause a match within the chain under analysis.

We will now see the two most common cases:

- Case 1

  When the pattern is as follows:

  **es(te|ta|to)**

  Here it is said that the prefix of the substrings to be accepted must be "es", while the suffix must be "te", "ta" or "to".

  And therefore the following substrings highlighted in yellow would be accepted:

  este 123 esta 456 esto

- Case 2

  When the pattern is as follows:

  **(?(regular_expression) yes | no)**

  where yes is an expression and no is another regular expression. This toggle construction works as follows based on the example below:

  **(?(\d)\d+|\x24%)**

If regular_expression is of type digits, then every time a digit is entered it will go through the expression yes(\d+), while if it is not met then the match will occur according to the expression of no (**\x24**%) which indicates that the substring must be a monetary sign ($) which corresponds to ASCII 24 hexadecimal followed by a percent symbol.

Below, the substrings that matched within the string under analysis are highlighted in yellow:

```
123.44 123 abc $%
```

It should be noted that the substring abc, although it does not meet the condition of being composed of digits only, does not meet the condition of not since it is not composed of $%.

If we modify the previous pattern by saying that decimal numbers must be included in the yes:

```
(?(\d)\d+\.\d+|\x24%)
```

then the matches would be including the point as shown below:

123.44123.44 abc $%

## 5. Brackets "[]" in more detail

The square brackets within regular expressions are of great importance because through them you can perform multiple actions that could algorithmically represent a lot of code if these expressions were not available.

They serve to specify, for example, a set of characters that may be present in a specific position of the string under analysis. It is also possible to specify ranges using the script as we saw in previous examples.

Inside square brackets almost all metacharacters lose their special meaning and are taken as normal characters; you only need to put a backslash before the backslash itself (written "\\"), before the closing bracket "]", before "^" if it is the first character and before the hyphen "-" if it is neither the first nor the last one. For example, [\\|] accepts a backslash or a vertical bar "|". Characters are not separated by commas either: [\\, |] would also accept the comma and the space. Within the square brackets, the period represents a literal character and not a special one, so it is not necessary to precede it with the backslash.

### 5.1 The + sign

The use of the + sign is widely used within regular expressions and means "one or more repetitions of the previous element." To make it clearer, some examples of its use are shown below:

1) Example:
Checking cases with a dot at the beginning without a sign:
Example: (.15, .972, etc.)
PATTERN:
**"^[.]\d+$"**

2) Example:
Checking cases with a dot at the end without a sign:
Example: (15., 972., etc.)
**"^\d+[.]$"**

3) Example:
Checking cases with sign and format {+,-}9(n).9(n):
Example: (+121.151, -45378.97277, etc.)
**"^[+-]\d+\.?(\d+)?$"**

4) Example:
Checking cases without sign and 9(n).9(n) format
and also of the unsigned integers 9(n)
Example: 459.15, 262.9725, 673, etc.)
**"^\d+\.?(\d+)?$"**

## 6. The parentheses "()"

Parentheses are used in regular expressions in a very similar way to square brackets, although with the following differences:

- They can be used to group alternatives, for example:

The expression "v1 (greater than|less than|equal to) v2" that will be seen below in an evaluative table illustrates the way parentheses are used:

**Example 1:**

***Important:***
***Substrings in yellow within the string under parsing mean they match with the regular expression, while unhighlighted characters mean they do not match. In C#, when cases similar to (4) and (5) occur, the complete string is given as ACCEPTED, however, when there is no substring that meets the condition, then the IsMatch method returns false.***

| Accepted Strings | Unaccepted Strings |
|---|---|
| 1) v1 equal to v2 | v1 equal to v3 |
| 2) v1 greater than v2 | v1 is greater than v2 |
| 3) v1 less than v2 | v1 is less than variable v2 |
| 4)v1 equal to v2 or v1 greater than v2 | v1 same as v2 or v1 greater than or equal to v2 |

| 5)v1 less than v2 or v1 greater than v2 | v1 unequal from v2 |
|---|---|

- In this other example the expression:

"^v1 (greater than|less than|equal to) v2$" restricts the possibilities of the previous expression because by inserting the characters ^ and $ at the beginning and end of the string we are saying that examples (4) and (5) of the table above would not be accepted.

- Parentheses can also be used to group special characters within them but without losing their meaning, unlike square brackets. Let's illustrate this concept with examples.

  Let's choose the following pattern:

  **(.A\d) which indicates that the accepted strings will be those that contain any character (the dot without a backslash represents any character), followed by an uppercase letter "A" (regular expressions are case-sensitive) and then a decimal digit. For a literal dot you would write \. and to also accept a lowercase "a", [Aa].**

**Example2:**

| Accepted Strings | Unaccepted Strings |
|---|---|
| 1) .A9 | .a9 (the lowercase "a" does not match "A") |
| 2) .A95 (contains .A9) | .b9 (does not have "A" as 2nd character) |
| 3) =A7 (the dot accepts any character) | A7 (the first character is missing) |

Let's now choose the previous pattern with brackets: [.A\d] (with no commas between the characters: [.,A,\d] would also accept the comma).

This pattern, however, matches every single character that is a dot, an uppercase letter "A" or a decimal digit, in any order in which they appear.

**Example3:**

**** Important:***
***It will be observed that the square brackets act differently from the parentheses even though both are grouping metacharacters, since in this example it is noted that the strings are not necessarily 3 characters long as in example 2, but can be strings of any number of elements as long as they contain points, uppercase letters A and decimal digits from 0 to 9. Of course, as long***

*as we do not use the left and right dimensions of the input string using the ^ and $ symbols at the beginning and end of pattern.*

| Accepted Strings | Why are they accepted? |
|---|---|
| .A95 | Contains the period, the letter "A" and digits |
| .a4444444444jklmxyz | Contains the period, the letter "A" and digits |
| 7666skl.bbb | Contains period and digits |
| .A999xyza5uuuu6912883 | Contains the period, the letter "A" and digits |
| 333,,,,,.#####aAAAAaFaa | Contains digits and the letter "A" and a period |

- The groupings in parentheses can sometimes represent a "tag" or "reference point" that are interpreted by the search engine in order to perform more advanced actions where they are combined with other special characters as will be seen later.

## 7. The character "."

The character "." represents any character in its place except \n and is used when you want the characters that follow it to agree in exactly the same order of appearance within the string under analysis, that is, if we choose (.er) as a pattern, it would accept chains like the following:

- comer
- hacer
- tener
- arder
- Si tengo que comer y cocer voy a tener que partirme en dos (If I have to eat and cook, I will have to split in two)

It will be observed that the substrings in yellow match the pattern, however, it is convenient to clarify that instead of the period there must be a character, therefore, the string er does not match the chosen pattern.
It is convenient that you compare the result obtained with the pattern (.er) and with the pattern [.er] and you will be able to see the differences between both options, we leave it to you.

If you wanted to also accept substrings, the pattern could be the following:

```
(.er|er) <== Using logical OR
```

and then strings like the following would be accepted:

ercomeraaatemerbbbsostener

## 8. The question mark "?"

The question mark in regular expressions can have several uses:

1. It can express optionality in the appearance of a character or a substring within the string under analysis, for example, if we choose the following pattern:

**(cen?tro)**

This would mean that the character preceding the ? symbol is optional within the string and therefore the substrings cetro or centro would be accepted.

****It is possible to nest the parentheses when you want to check more than one character before the ? character. In the following example, strings that have "nxyz" or nothing between ce and tro are accepted.***

```
Pattern ==>(ce(nxyz)?tro)
String in analysis ==>cenxyztroaaaaacetrobbbbb
```

2. It is also possible, based on the previous example, to search for substrings of more than one character, for example:

   **Sept(iembre|ember)?**

   this would mean that the substrings iembre or ember, which are separated by the logical OR ("|") and appear within the parentheses that precede the ? sign, would be optional within the string.

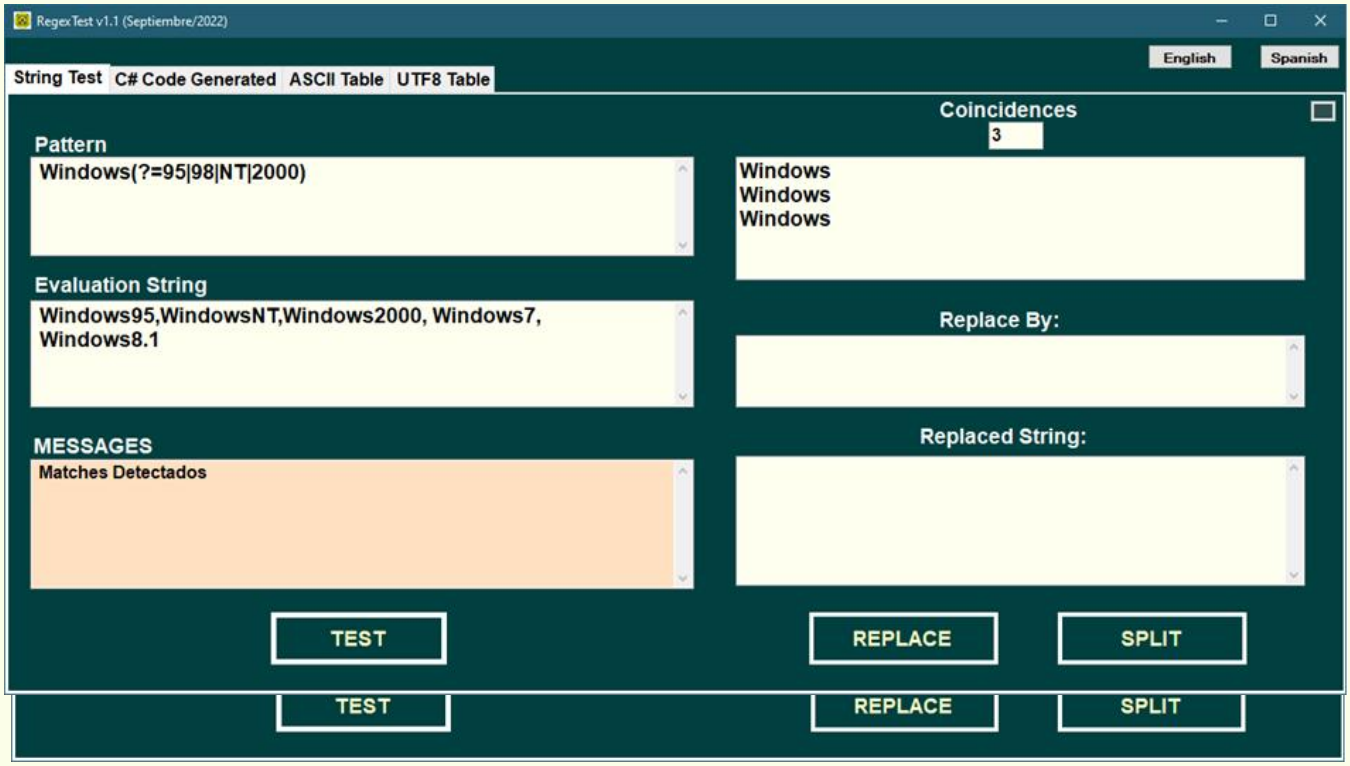
   For this reason the substrings Sept, Septiembre or also September would be accepted by the pattern. Of course, substrings such as Noviembre or December would not be accepted.

## 9. The format (?:pattern)

This format groups a part of the pattern without keeping it as a captured group (that is why it is called a non-capturing group) and it is often used linked to the "|" (OR).

If the following pattern is chosen: industr(?:y|ies), this would be a simplified form compared to this other equivalent pattern: industry|industries.

If we process this pattern under our RegexTest utility it would give us the following result:




## 10. The format (?=pattern)

This format is known as a lookahead: it checks that what is written inside the parentheses comes right after the match, but without including it in the match. It is often used linked to the "|" (OR).

For example:
Pattern ==> Windows(?=95|98|NT|2000)
Makes the following string have three matches (in yellow):
Windows95,WindowsNT,Windows2000, Windows7, Windows8.1

*** It will be noted that Windows7 and Windows8.1 do not have coincidences.
In RegexTest it would appear like this:

## 11. The format (?!pattern)

It is the negation of the previous one, therefore, if it were written:
Windows(?!95|98|NT|2000)
It causes the following string to have two matches, which are exactly the ones that were not accepted in the example in the previous section:

Windows95,WindowsNT,Windows2000,Windows7,Windows8.1
Example:

## 12. The Quantifiers

Although we have already seen it previously, we can summarize thatthe symbols:

**?**, *, +, {n,m} or {n,}

They are known as quantifiers in regular expressions.

Below is an illustrative summary of different cases as examples and seen in more detail:

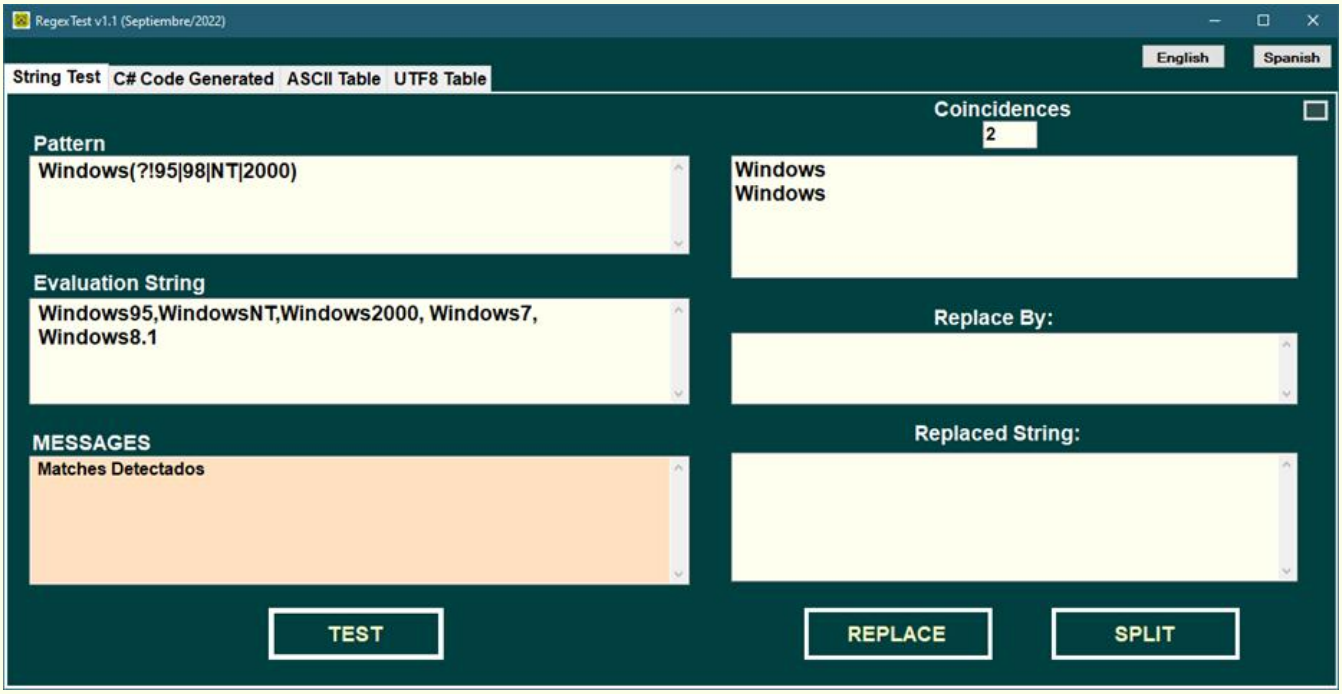
Expansive and non-expansive quantifiers

* According to Microsoft:

Quantifiers specify how many instances of a character, group, or class of characters there must be in the input for a match to be found. The following table lists the quantifiers supported by the .NET Framework.

The quantities n and m are constants of integer type. Typically, quantifiers are expansive and cause the regular expression engine to match as many occurrences of particular patterns as possible.

When appending the character ? a quantifier makes it non-expansive and makes the regular expression engine match as few occurrences as possible.




| Expansive Quantifier | Non-expansive quantifier | Description |
|---|---|---|
| * | *? | Search for zero or more matches. |
| + | +? | Search for one or more matches. |
| ? | ?? | Search for zero or one match. |
| {n} | {n}? | Searches for exactly n matches. |
| {n,} | {n,}? | Search for at least n matches. |

| | | |
|---|---|---|
| {n,m} | {n,m}? | Search for nam matches. |

### Some examples of the use of quantifiers (Microsoft)

| Quantificator | Description | Model | Coincidents |
|---|---|---|---|
| * | Indicates that there can be 0 or more occurrences of digits before the decimal point and a single digit to the right of the point. | "\d*\.\d" | ".5" accepts ".5"<br>"59.66" accepts "59.6"<br>"473.833.4" accepts "473.8" and "33.4"<br>(2 matches) |
| + | Indicates that there must be 1 or more occurrences of digits before the decimal point and a single digit to the right of the point. 0 occurrences are not accepted as in the previous case (*) | "\d+\.\d" | ".5" is not accepted<br>"1.5" if accepted |
| {n} | Coincides with the previous element exactly n times. | ",\d{3}" | ",043,958,317" gives 3 matches |
| {n,} | Coincides with the previous pattern(\d) at least<br>n times. | "\d{2,}" | "166", "29", "1930" |
| {n,m} | Coincides with the previous pattern (\d) at least<br>n times, but not more than m times. | "\d{3,5}" | "166", "17668","19302" in "193024" |
| *? | Coincid with the previous pattern (\d) zero or more times, but as few times as possible. | "\d*?\.\d" | ".0", "19.9", "219.9" |
| +? | CoinIt occurs with the previous pattern (be)one or more times, but as few times as possible. | "be+?" | "be" in "baby" |
| {n}? | Coincides with the previous pattern (\d)exactly n times. | "\d{3}?" | ".434" in ".434"<br>".434" in ".43488" |
| {n,}? | Coincides with the previous pattern (\d) at least n times, but as few times as possible. | "\d{2,}?" | "166" with 1 match<br>"1930" with two coincidences |
| {n,m}? | Coincides with the previous pattern (\d) | "\d{2,5}?" | "19","30", and "24" in "193024" |

| | | | |
|---|---|---|---|
| | between n and m times, but as few times as possible. | | (3 matches) |

| Pattern | Permitted data |
|---|---|
| a*x | x, ax, aax, aaax... |
| a?x | ax, x |
| a+x | ax, aax, aaax... |
| (a\|b)+x | ax, bx, aax, abx, bax, bbx, aaax, aabx, abax, abbx, baax, babx, bbax, bbbx, aaaax... |

## 13. Regular Expression Utilities

The RegexTest utility has been programmed entirely in C# language as a Windows Forms type application and for this reason all the examples given in this document have been tested under this utility in order to avoid incompatibilities that may exist with respect to other platforms, if If the reader wants to try regular expressions on other platforms, we suggest that they try RegexBuddy, which compared to other similar products, looks quite robust and although it is shareware, it is not offered at a very high price.

Although we have previously shown the results referring to the RegexTest screen, its fundamental characteristics will be shown below.

The main screen is conceived under a multi-tab format where in the first TAB all the parameters that are part of the validation to be carried out are given (pattern, string to analyze (string), the number of matches detected (matches). ), by whom you want to replace the input string, the replaced string in cases of replacement, and the division of a string into substrings or SPLIT).

In this TAB there are three buttons:

- **Chain Test**
- **Replace**
- **Split**

After performing the string test, you can proceed to perform replacements or division of strings into specific substrings (split).

## 14. The Replace method of the **Regex** class.

So far we have seen different cases for finding substrings within a given string that are accepted by a given pattern (Regex IsMatch method), however, it is also possible to replace matching substrings with a given string, and this is done precisely through the Replace method of the Regex class that will be explained below.

**Replace method**

This method has several overloads that can be seen in the Visual Studio Help, but here are two frequently used ones:

```
1)Regex.Replace(string Chain,string Pattern,string ReplaceBy);
2)OBJ.Replace(string Chain,string ReplaceBy);
```

In the first case, it is not necessary to instantiate any object since the static Replace method of the Regex class is being used.

In the second case it is assumed that the OBJ object has been previously instantiated with new:

```
Regex OBJ = new Regex(@"validation pattern");
```

Both return a string as a result of replacing the substrings that have been Matched with the pattern given by the string of the ReplaceBy variable.

For example, if we assume that our pattern is:

```
[\x61-\x7a]{3}?
```

which accepts all three-element substrings containing lowercase letters from "a" to "z", and the string we give is:

```
aaa111bbb222ccc333ddd
```

where the substrings that respond are the letters highlighted in yellow, then if we put:

```
string R =Regex.Replace("aaa111bbb222ccc333ddd",
                        "[\x61-\x7a]{3}?",
                        "ZZZ");
```

In R the following would remain:

R = "ZZZ111ZZZ222ZZZ333ZZZ"

Let's see below an example with a new case of a pattern that appears frequently and that is the detection of spaces, which is done using the \s escape sequence.

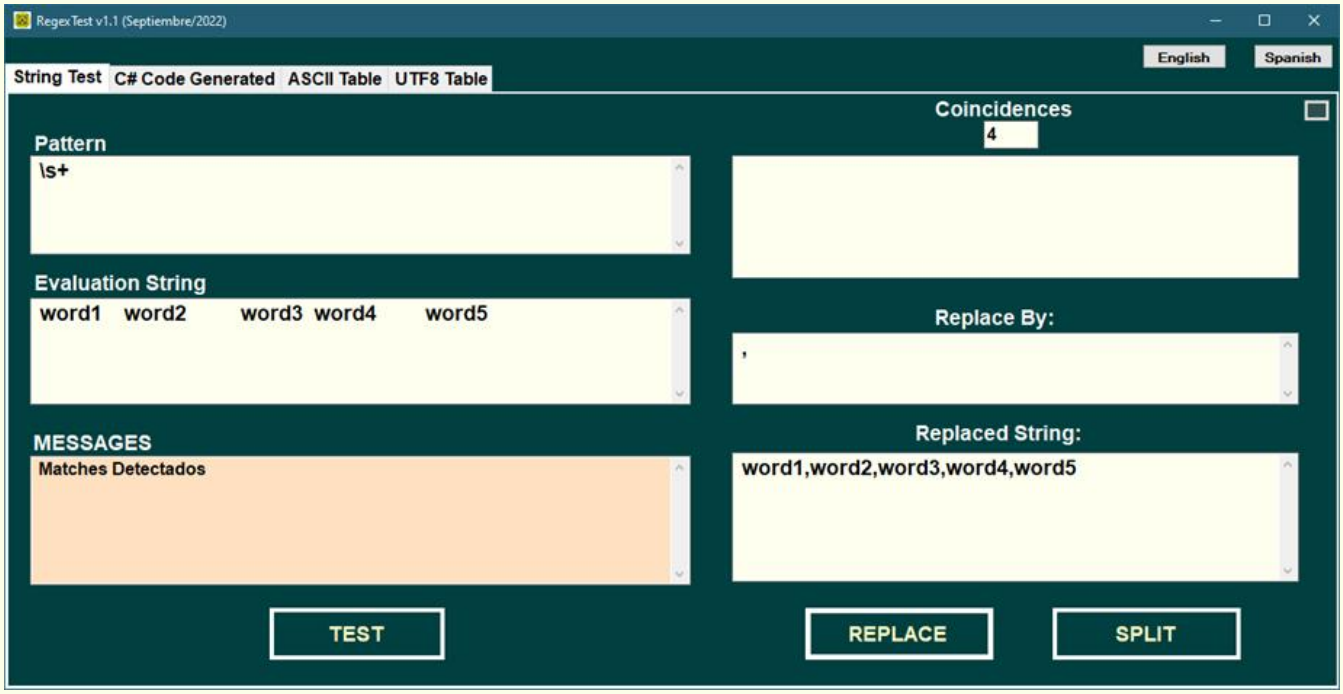
In the following example we want to eliminate all the extra spaces that separate word1, word2, etc.

The first thing we must do is define our pattern as follows:

**\s+**which means that one or more spaces will be detected where they are found (remember the argument + which means one or several times as opposed to * which means zero or several times).

Then we give RegexTest the string to evaluate that contains multiple spaces and we immediately touch the String Test button, resulting in 4 matches that do not appear reflected under Matches because they are spaces.

To perform a replacement, after having carried out the test, define what the character or string that will replace the detected matches will be, which in our case is the comma, and when you click on the Replace button the result will remain. in the Replaced Chain box.




If after having carried out all the previous actions we want to see the C# code corresponding to them, then this is perfectly possible using the RegexTest utility and is also very simple, as just touch the second TAB called Generated C# Code and it will appear in a box editable all the generated code that corresponds to these actions as follows:

```
//============================================================ ==========================
// IsMatch() code
//============================================================ ==========================
      if (Regex.IsMatch(@"word1 word2 word3 word4 word5",@"\s+"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the amount is obtained in the CoincidenciasN variable
// of Matches that were obtained

int Coincidencias1 = Regex.Matches(@"word1 word2 word3 word4 word5",@"\s+").Count;
//============================================================ ==========================
// Replace() code
//============================================================ ==========================
string ReplacedString1 = Regex.Replace(@"word1 word2 word3 word4 word5",@"\s+",@",");
//============================================================ ==========================
```

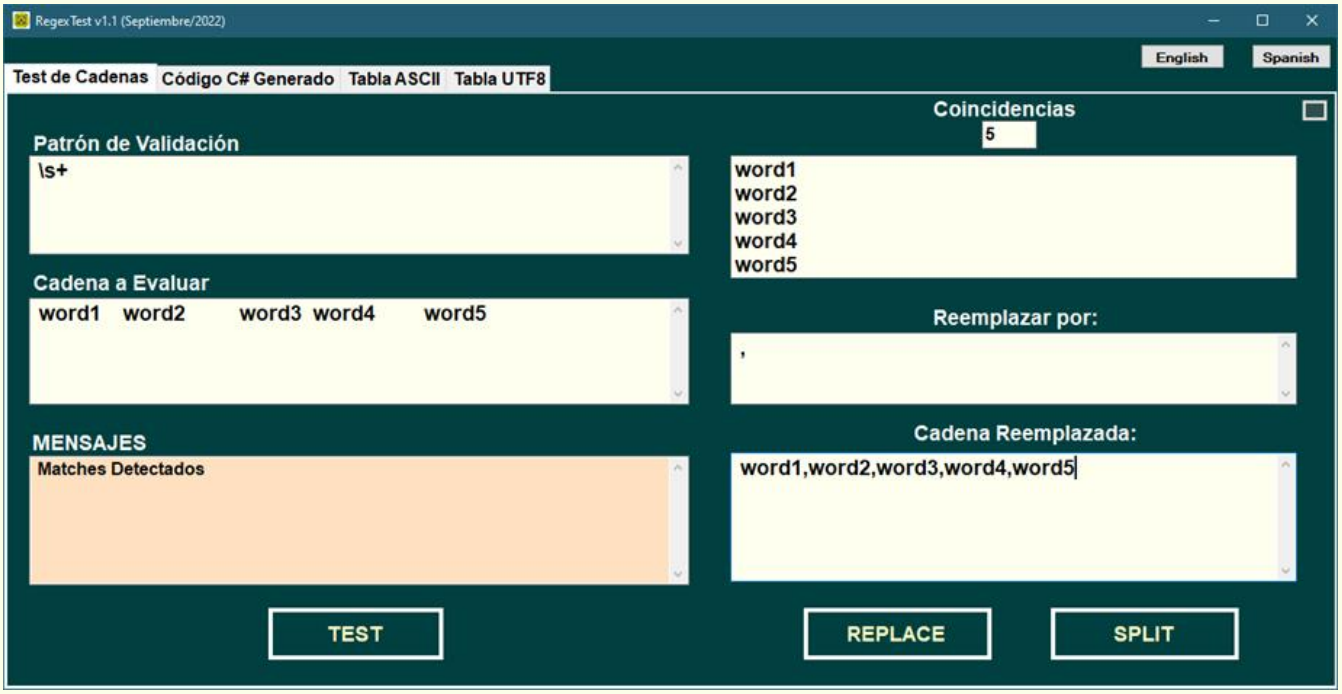
As can be seen, the programmer is left with correctly generated C# code and can copy and paste it into his program, making the adjustments he wants very easily but without the fear that it may present syntax errors in his program.

Now, if instead of replacing you had wanted to make a separation of all the words in the example (word1, word2, etc.), whose separator is in our case the original spaces that were changed by commas, then it would be enough to touch the button Split generating the following code:

```
//============================================================ ==========================
// Split() code
//============================================================ ==========================
// Below are the substrings that are between the
// substrings that matched in the MATCH assuming that these are the
// delimiters within the string being parsed.
//============================================================ ==========================
string[] StringSplit1 = Regex.Split(@"word1 word2 word3 word4 word5",@"\s+");
for (int IDX = 0; IDX < StringSplit1.Length; IDX++)
{
listBox1.Items.Add(StringSplit1[IDX]);
}
//============================================================ ==========================
```

The main RegexTest screen would appear as follows:

After having seen the process of the previous cases, we must emphasize clarifying the great




advantage of the RegexTest utility and that is that to the extent that these previous cases were executed, in the TAB called: **Generated C# Code**

The program code corresponding to each case appears, which is generated dynamically as we click on TEST, REPLACE or SPLIT as shown below:

```
//======================================================================
// USING's for Regex
// < Generated Code By RegexTest >
//======================================================================================
// The code generated by RegexTest shown below has as objective to provide
// you with a template that can be easily modified by you and be inserted
// in any region of your program.
// This template assumes the existence of two textBox named:
// - STRING.Text
// - PATTERN.Text
// It also assumes the existence of a listBox named:
// - <listBox1>
// For this reason, if you have the string to parse in a variable of type
// string, like the pattern, it will suffice to do a simple search with
// replacement so that the implicit names of the textBox above are replaced
// by the names of the variables you want
//======================================================================================
using System.Text.RegularExpressions;
//======================================================================================
```

```
//=============================================================================================
// IsMatch() Code
//=============================================================================================
if (Regex.IsMatch(@"industria industries industry industries",@"industr(?:y|ies)"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Coincidencias1 = Regex.Matches(@"industria industries industry;
industries",@"industr(?:y|ies)").Count;
//=============================================================================================
// IsMatch() Code
//=============================================================================================
if (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Coincidencias2 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)").Count;
//=============================================================================================
// IsMatch() code
//=============================================================================================
if (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Coincidencias3 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?=95|98|NT|2000)").Count;

//=============================================================================================
// IsMatch() code
//=============================================================================================
if (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Coincidencias4 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)").Count;
```

```
//=============================================================================================
// IsMatch() code
//=============================================================================================
if (Regex.IsMatch(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Matches5 = Regex.Matches(@"Windows95,WindowsNT,Windows2000, Windows7,
Windows8.1",@"Windows(?!95|98|NT|2000)").Count;
//=============================================================================================
// IsMatch() code
//=============================================================================================
if (Regex.IsMatch(@"word1 word2 word3 word4 word5",@"\s+"))
{
// Put here code if the match IS FULFILLED
}
else
{
// Put code here if the match IS NOT MET
}
// Next, the Obtained Matches are obtained in the CoincidenciasN variable
int Matches6 = Regex.Matches(@"word1 word2 word3 word4 word5",@"\s+").Count;
//=============================================================================================
// Replace() code
//=============================================================================================
string ReplacedString1 = Regex.Replace(@"word1 word2 word3 word4 word5",@"\s+",@",");
//=============================================================================================
//=============================================================================================
// Split() code
//=============================================================================================
// Split() method extracts all substrings not contained in substrings
// of matches
//=============================================================================================
string[] StringSplit1 = Regex.Split(@"word1 word2 word3 word4 word5",@"\s+");
for (int IDX = 0; IDX < StringSplit1.Length; IDX++)
{
listBox1.Items.Add(StringSplit1[IDX]);
}
//============================================================== ============================
```

**The advantage of having this utility is that it eliminates the possibility of the programmer making syntax errors in his program, by having the possibility of inserting the generated code in the places he wants.**

## 15. Excluding Character Groups from a Character Class

These expressions are used to remove or exclude a set of characters from a character class, the format used is the following:

[base_group-[excluded_group]]

where square brackets are required and hyphens are required in the expression.

Example:

[0-9-[01]]

This pattern would accept the following substrings, since neither zero nor one is accepted:

1234 000 544 666 999 111

## 16. Exercises to Solve

Below are a series of practical exercises that will help you assimilate regular expressions in C#.

It is advisable that you use RegexTest to validate results.

### 16.1 - Exercise 1

Create a regular expression pattern that accepts First and Last Names in Spanish. (They may contain stressed vowels and the ñ. The first letter must be capitalized)

### 16.2 - Exercise2

Create a regular expression pattern that accepts decimal numbers of at most five whole digits and two decimal digits, without a sign.

### 16.3 - Exercise3

Create a regular expression pattern that accepts decimal numbers of at most five integer digits and two mandatory decimals, eventually signed.

### 16.4 - Exercise4

Create a regular expression pattern that accepts phone numbers with the following format:

ddd-ddd-dddd

### 16.5 - Exercise5

Create a regular expression pattern that accepts only blank lines.

### 16.6 - Exercise6

Create a regular expression pattern that accepts dates with the following format:

dd/mm/yyyy

It is assumed that the days must be in the interval from 1 to 31, the months from 1 to 12 and the year must be between 1900 and 2099. (Do not analyze whether the year is a leap year or the months of 30 days as this normally requires of a small algorithm)

### 16.7 - Exercise7

Create a regular expression pattern that accepts words that represent verbs in the Spanish language (ar, er or ir ending).

### 16.8 - Exercise8

Create a regular expression pattern that accepts an identification number of a work center whose formation law is the following:

- The first two characters must be lowercase letters "a" through "z."
- A script follows.
- Finally there must be 5 decimal digits that must be in the range 4 to 8.

For example: "ab-47856"

### 16.9 - Exercise9

Create a regular expression pattern that accepts strings containing triplets of lowercase letters.

It is recommended that you consult the Visual Studio Help by searching for "Regular Expressions" and you will find multiple examples and concepts that will help you delve deeper into this topic.

## Final summary

The main objective of what we have seen so far has been to give an elementary vision of regular expressions and has been entirely oriented towards programmers not familiar with the subject.

Although this document has been intended only for programmers of the Microsoft C# language, it can also be useful for any programmer of other .NET languages, such as, for example, Visual Basic, C++, etc., and even other platforms and languages. This is because the basic ideas of regular expressions are common to many programming languages, text editors, etc., although each one has small differences.

If the reader wishes to continue delving into advanced topics of regular expressions, it is advisable that they study the Microsoft Help for Visual Studio in order to assimilate the different concepts taught here.

We hope that this introductory document has been useful to you, enjoy!

## *Note from the author:*

I apologize to the reader of the English document as I have not had enough time to do a thorough review of this document.